

Day 6 Evidence Workbooklet — Instructor Key

Branch Outcomes with if and else

Engineering practice → condition result → branch followed → selected action → support plan

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

Name		Section		Date	
Score	____/42		Mastery check	secure / developing / needs support	

The Week 2 scenario starts with a sensor-kit checkout table. A kit may be released, charged, sent to calibration, or held for missing checkout information. Today the focus is a single branch decision: read one condition, choose the branch that runs, and explain the action selected.

Engineering task	Decide which checkout action a sensor kit should receive from one condition at a time.
Computational move	Turn a true/false condition into one selected branch outcome while keeping the evidence for that branch outcome visible.
Python focus	<code>if</code> , <code>else</code> , condition result, selected branch, action variable, and printed branch outcome evidence.
Evidence to keep	case value, condition expression, <code>True/False</code> result, branch followed, action assigned, and support note.

Day 6 working rules

1. Python checks the condition after `if`.
2. If the condition is `True`, the `if` branch runs.
3. If the condition is `False`, the `else` branch runs.
4. Only one of those two branches runs.
5. A reliable branch note names both the condition result and the selected action.

1 Read the checkout condition before choosing an action

[recognize](#)[predict](#)

The lab team is checking whether a sensor kit can leave the checkout table. Day 6 uses one decision at a time: if the condition is true, one action happens; otherwise the other action happens.

```
1 kit_label = "S-14"
2 charge_percent = 82
3 charge_minimum = 80
4 calibration_label = "current"
5 borrower_initials = "JL"
6
7 charge_ok = charge_percent >= charge_minimum
8 calibration_ok = calibration_label == "current"
9 borrower_logged = borrower_initials != ""
10
11 if charge_ok:
12     action = "send to calibration check"
13 else:
14     action = "charge before checkout"
```

Pts.	Prompt	My evidence
1	Which variable stores the kit label?	Key: <code>kit_label</code> .
1	Which comparison creates <code>charge_ok</code> ?	Key: <code>charge_percent >= charge_minimum</code> ; here that is <code>82 >= 80</code> .
1	Is <code>charge_ok</code> true or false for this kit? Show the numbers.	Key: <code>True</code> ; <code>82 >= 80</code> .
1	Which branch assigns the action for this kit?	Key: The <code>if</code> branch runs because <code>charge_ok</code> is <code>True</code> ; it assigns <code>send to calibration check</code> .

2 Trace an if/else branch outcome

[trace](#) [explain](#)

Complete the branch outcome table. Use the condition result to decide which branch runs.

Pts.	Charge value	Condition	Branch followed	Action and reason
2	82	charge_percent >= 80	Key: see reason	Key: Result True; follow the if branch and set action to send to calibration check because 82 >= 80.
2	80	charge_percent >= 80	Key: see reason	Key: Result True; follow the if branch and set action to send to calibration check; equality is included by >=.
2	79	charge_percent >= 80	Key: see reason	Key: Result False; follow the else branch and set action to charge before checkout because 79 < 80.
2	Explain why the exact value 80 matters.	Key: 80 is the boundary. With >=, 80 >= 80 is True, so the exact minimum follows the if branch.		

Boundary connection

The branch outcome depends on a comparison from Week 1. If the comparison is wrong, the selected branch outcome will be wrong too.

3 Choose the selected action from printed evidence

[predict](#)[explain](#)

The same structure is used for calibration status. Read the code and predict the action.

```
1 calibration_label = "expired"
2 calibration_ok = calibration_label == "current"
3
4 if calibration_ok:
5     checkout_status = "ready for borrower log"
6 else:
7     checkout_status = "send to calibration bench"
8
9 print(kit_label, checkout_status)
```

Pts.	Prompt	My evidence
2	What value is stored in <code>calibration_ok</code> ? Why?	Key: <code>False</code> , because <code>calibration_label</code> is <code>expired</code> , not <code>current</code> .
2	Which branch runs: the <code>if</code> branch or the <code>else</code> branch?	Key: The <code>else</code> branch runs because <code>calibration_ok</code> is <code>False</code> .
2	What exact status text is printed after the kit label?	Key: <code>send to calibration bench</code> .
2	Why would <code>bool(calibration_label)</code> be unsafe here?	Key: Any nonempty label such as <code>expired</code> is truthy, so it could look acceptable even though the needed value is exactly <code>current</code> .

Complete the condition so the kit moves to borrower logging only when the borrower initials are present.

```
if _____:
    next_step = "record borrower and release kit"
else:
    next_step = "hold until borrower initials are entered"
```

Pts.	Prompt	My response
3	Write the completed condition using <code>borrower_initials</code> .	Key: <code>borrower_initials != ""</code> .
2	Why should the condition compare against an empty string instead of checking the kit label?	Key: The branch is about whether borrower initials were entered. A kit label can exist even when <code>borrower_initials</code> is empty.
2	Give one input value that should follow the <code>else</code> branch.	Key: <code>borrower_initials = ""</code> should follow the <code>else</code> branch.
1	Name the evidence type you used: state / type / boundary / branch.	Key: branch.

5 Debug a branch-outcome claim

[debug](#) [test](#) [explain](#)

A teammate writes: "The kit is ready because `charge_ok` is true." Use branch-outcome evidence to evaluate that claim.

Pts.	Prompt	My response
2	What part of the claim is supported by the code?	Key: The charge evidence is supported: <code>charge_ok</code> is <code>True</code> because <code>82 >= 80</code> .
2	What evidence is still missing before a full checkout release?	Key: Need calibration and borrower evidence, such as <code>calibration_ok</code> being <code>True</code> and <code>borrower_initials</code> not being empty.
2	Give one test case that should follow the hold or repair outcome.	Key: Example: <code>charge_percent = 82</code> , <code>calibration_label = "expired"</code> , and <code>initials</code> present should produce calibration support, not release.
2	Write a safer one-sentence checkout note.	Key: The kit has enough charge, but release should wait until calibration is <code>current</code> and borrower initials are present; otherwise use the matching support plan.

Pts.	My checkout-decision note
4	Write a short note that says how an <code>if/else</code> statement chooses one action from one condition. Use the sensor-kit checkout example. Key: An <code>if/else</code> statement checks one condition and runs exactly one branch. For charge, <code>82 >= 80</code> is <code>True</code>, so the <code>if</code> branch chooses <code>send to calibration check</code>; a low charge would choose <code>charge before checkout</code>.
2	Circle the support plan you would use next: condition result / branch followed / exact boundary / string label / written explanation. Name one next action: _____

Bridge to Day 7

Day 6 chooses between two actions. Day 7 adds ordered rules where the first true condition wins and later branches are skipped.

VIP Lab Alignment

Bridge Day 6 • Contract 2d4127aac856

This page replaces the earlier wrapper-based lab bridge and follows the current top-level notebook interface.

Why this page is here

VIP Lab Day(s): 3

Activities: 18.8, 18.9, 18.10, 18.11, 18.12

Finish the current calculation-and-output lab with top-level code cells; do not use or author a function wrapper.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.8 18-8-work / 18-8-transfer / 18-8-cold	Badge Sheet Decision	Calculate a total from two named quantities.
18.9 18-9-work / 18-9-transfer / 18-9-cold	Simple Statistics	Use named results for minimum, maximum, total, and average.
18.10 18-10-work / 18-10-transfer / 18-10-cold	Box Model Values	Translate the volume and surface-area formulas into named calculations.
18.11 18-11-work / 18-11-transfer / 18-11-cold	Structured Text Shape	Treat characters and line order as exact output requirements.
18.12 18-12-work / 18-12-transfer / 18-12-cold	Storage Bay Estimate	Translate a rectangular-volume estimate into one expression.

Readiness check before you code

- For Activity 18.8, write the two exact boundary values that must be tested before you open the notebook.

Answer and explanation: The exact decision boundaries are 8 and 16. Test values immediately below, at, and immediately above each boundary.

- Choose one Activity 18.9-18.12 and name the intermediate value and exact displayed result you will inspect first.

Answer and explanation: Accept any activity-specific answer that names a real intermediate quantity and its required output, such as total and average for 18.9 or volume and surface area for 18.10.

Notebook and submission procedure

- Use the sample and transfer cells for formative practice; attempt before opening a reveal.
- Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only the Studio cells listed above are scored for independent mastery in the matching Lab Day notebook.
- Save or download the completed notebook as one `.ipynb` file.
- Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.